Computação Distribuída com MPI

Prof Alcides/Prof Mario

MPI + Python + Docker

Detecção distribuída de áreas suspeitas de Pneumonia em radiografias de pulmão com MPI

Durante períodos de alta demanda hospitalar, grandes redes de saúde precisam analisar rapidamente um grande volume de radiografias de tórax para identificar casos suspeitos de alterações pulmonares.

Uma radiografia digital pode ser tratada computacionalmente como uma matriz de pixels em tons de cinza, em que:

- valores baixos representam regiões mais escuras
- valores altos representam regiões mais claras
- determinadas áreas claras em regiões pulmonares podem indicar opacidades, inflamações ou outras alterações que merecem atenção clínica

Em sistemas reais, a análise automatizada dessas imagens pode ser usada como triagem inicial, ajudando equipes médicas a priorizar exames que exigem avaliação mais urgente.

Como hospitais e centros de pesquisa lidam com milhares de exames, é comum usar processamento paralelo e distribuído para acelerar etapas de análise. Neste exercício, você deverá simular esse cenário utilizando MPI em Python.

Problema

Você foi contratado para desenvolver um protótipo de um sistema distribuído que analisa radiografias de pulmão em um cluster computacional.

O sistema deverá:

1. gerar ou carregar uma radiografia simulada;
2. dividir a imagem entre os processos do cluster;
3. analisar partes da imagem em paralelo;
4. consolidar os resultados;
5. gerar um relatório final com métricas globais;
6. comparar o comportamento da aplicação em diferentes execuções.

O foco deste projeto é a comunicação distribuída com MPI e a capacidade de interpretar o que acontece durante a execução paralela.

Objetivo

Desenvolver um programa em Python com mpi4py que utilize, obrigatoriamente:

- MPI_Bcast
- MPI_Scatter
- MPI_Reduce
- MPI_Gather
- MPI_Barrier

para realizar uma análise distribuída de uma radiografia digital simulada.

Interpretação dos dados da imagem

A radiografia será representada como uma matriz 2D.

Cada posição da matriz representa um pixel da radiografia.

Exemplo:

[32, 35, 40, 38, 36]
[45, 48, 180, 210, 205]
[41, 44, 190, 220, 215]
[39, 42, 60, 55, 50]

Como interpretar

- valores entre 0 e 255
- 0 representa preto
- 255 representa branco
- regiões muito claras podem representar áreas densas ou suspeitas

Neste projeto, você irá considerar que:

- pixels com intensidade muito alta podem indicar áreas suspeitas
- agrupamentos de pixels muito claros podem indicar regiões de atenção clínica

Importante: este exercício não realiza diagnóstico médico real, trata-se de uma simulação.

O que cada parte da matriz representa

Considere que a radiografia simulada representa o tórax dividido em duas grandes áreas:

- metade esquerda da imagem → pulmão esquerdo

- metade direita da imagem → pulmão direito

Além disso:

- cada linha da matriz representa uma faixa horizontal da radiografia
- cada coluna representa uma posição lateral da imagem
- cada pixel representa um ponto da imagem em escala de cinza

Assim, quando um processo recebe um conjunto de linhas da matriz, ele está analisando uma faixa horizontal da radiografia, como se estivesse examinando uma porção do exame.

Requisitos técnicos obrigatórios

Seu programa deve utilizar todos os mecanismos abaixo:

1. Broadcast

Usado para enviar os dados iniciais a todos os processos.

2. Scatter

Usado para distribuir as partes da radiografia entre os processos.

3. Reduce

Usado para consolidar valores numéricos globais.

4. Gather

Usado para enviar estatísticas locais de cada processo ao processo root.

5. Barrier

Usado para sincronização entre as etapas do processamento.

Etapas do projeto

Etapa 1 – Inicialização do ambiente MPI

Todos os processos devem inicializar o ambiente MPI e descobrir:

- seu identificador (rank)
- a quantidade total de processos (size)

Exemplo esperado no seu raciocínio:

- rank 0 → coordenador/root
- ranks 1, 2, 3... → trabalhadores

Nesta aplicação, o processo root será responsável por preparar os dados e consolidar o relatório final.

Etapa 2 – Geração da radiografia simulada no processo root

Somente o processo rank 0 deve gerar a radiografia.

Tamanho sugerido

Você pode começar com algo como:

- 2000 x 2000 pixels

Depois testar tamanhos maiores, como:

- 4000 x 4000
- 6000 x 6000

Faixa de valores

Cada pixel deve ter intensidade entre:

- 0 e 255

Como simular uma radiografia mais interessante

Não gere apenas números totalmente aleatórios sem padrão. Tente construir algo mais coerente com a proposta:

- fundo externo do corpo com intensidades diferentes;
- região central do tórax com valores intermediários;
- pulmões com intensidades mais escuras em média;
- algumas manchas claras artificiais simulando áreas suspeitas.

Mesmo que simplificado, isso dá mais sentido ao projeto.

Etapa 3 – Definição dos parâmetros de análise

O root deve definir os parâmetros da análise, por exemplo:

- limiar de suspeita leve
- limiar de suspeita alta
- percentual mínimo para classificar uma faixa como crítica

Exemplo:

- pixel > 200 → suspeito
- pixel > 230 → altamente suspeito

Esses valores devem ser enviados para todos os processos com MPI_Bcast.

O que deve ser enviado com broadcast

Envie pelo menos:

- número de linhas da imagem

- número de colunas da imagem
- limiar de suspeita leve
- limiar de suspeita alta
- percentual mínimo de pixels suspeitos para classificar uma faixa como crítica

Exemplo:

Suponha que os parâmetros definidos sejam:

- limiar de suspeita leve: 200
- limiar de suspeita alta: 230
- percentual crítico: 5%

Nesse caso:

- pixels com intensidade maior que 200 serão considerados suspeitos
- pixels com intensidade maior que 230 serão considerados altamente suspeitos
- se mais de 5% dos pixels analisados por um processo forem suspeitos, aquela faixa da radiografia poderá ser classificada como crítica

Etapa 4 – Sincronização inicial com MPI_Barrier

Após o broadcast dos parâmetros, use MPI_Barrier() para garantir que todos os processos:

- já receberam as configurações
- estão prontos para iniciar a etapa de análise

Aqui o barrier faz bastante sentido porque marca o início sincronizado da execução analítica.

Etapa 5 – Divisão da radiografia com MPI_Scatter

A radiografia deve ser dividida por faixas horizontais.

Exemplo

Imagem com 2000 linhas e 4 processos:

- processo 0 → linhas 0 a 499
- processo 1 → linhas 500 a 999
- processo 2 → linhas 1000 a 1499
- processo 3 → linhas 1500 a 1999

Cada processo deverá receber apenas sua parte da imagem. Registre esta informação para usar posteriormente.

Atenção importante

Pense no que fazer quando o número de linhas não for divisível exatamente pelo número de processos.

Essa parte pode ser tratada de duas formas:

- ajustar o tamanho da imagem para facilitar os testes;
- ou implementar uma lógica de preenchimento/redistribuição.

Essa decisão faz parte do desafio do projeto.

Etapa 6 – Análise local de cada processo

Cada processo deve analisar sua faixa da radiografia e calcular métricas locais.

Cada processo deve calcular, no mínimo:

1. quantidade de pixels analisados
2. soma total das intensidades
3. média local de intensidade (utilizada apenas para relatório local)
4. maior intensidade local
5. quantidade de pixels suspeitos
6. quantidade de pixels altamente suspeitos
7. quantidade de pixels suspeitos no pulmão esquerdo
8. quantidade de pixels suspeitos no pulmão direito

Como identificar pulmão esquerdo e direito

Dentro da faixa recebida:

- colunas da metade esquerda da imagem → pulmão esquerdo
- colunas da metade direita → pulmão direito

Assim o processo não analisa só intensidade; ele também interpreta a posição da alteração.

Etapa 7 – Classificação local da faixa analisada

Além das métricas numéricas, cada processo deve classificar sua faixa da radiografia.

Exemplo de classificação da faixa

- normal → poucos pixels suspeitos
- atenção → quantidade moderada de pixels suspeitos
- crítica → alta concentração de pixels suspeitos ou altamente suspeitos

A regra de classificação deve ser definida por você, mas precisa ser:

- objetiva

- documentada
- coerente com os dados gerados

Isso obriga você a pensar sobre os critérios da análise.

Etapa 8 – Simulação de cluster heterogêneo

Para tornar o projeto mais realista, introduza uma simulação de diferença de desempenho (use sleep) entre os nós.

Por exemplo:

- processos de rank ímpar podem ter atraso artificial;
- ou processos com rank maior que 0 podem ter atrasos variáveis.

A ideia é representar que, em um cluster real, nem todas as máquinas têm exatamente o mesmo desempenho.

Essa simulação deve ser feita depois do processamento local dos processos, mas antes da consolidação final.

Etapa 9 – Sincronização antes da consolidação com MPI_Barrier

Antes de consolidar os resultados, use novamente MPI_Barrier().

Garantindo que todos os processos terminaram suas análises locais antes de partir para:

- redução global
- coleta de estatísticas
- geração do relatório final

Essa etapa é importante para você perceber o efeito do processo mais lento sobre o tempo total da aplicação.

Etapa 10 – Consolidação numérica com MPI_Reduce

Agora os processos devem consolidar os resultados numéricos globais usando MPI_Reduce.

Use Reduce para obter pelo menos

- total global de pixels analisados
- soma global das intensidades (usada para calcular a média global)
- total global de pixels suspeitos
- total global de pixels altamente suspeitos
- total de suspeitos no pulmão esquerdo
- total de suspeitos no pulmão direito

- maior intensidade global

A partir disso, o processo root poderá calcular

- média global da imagem
- percentual global de área suspeita
- distribuição de suspeitas por lado do pulmão

Etapa 11 – Coleta de estatísticas detalhadas com MPI_Gather

Além da redução numérica, cada processo deve montar um pequeno relatório local e enviá-lo ao root usando MPI_Gather.

Cada relatório local deve conter, por exemplo

- rank do processo
- intervalo de linhas analisadas
- total de pixels analisados
- quantidade de suspeitos
- quantidade de altamente suspeitos
- maior intensidade local
- classificação da faixa analisada (normal, atenção ou crítica)

Esses dados serão usados pelo root para compor um relatório detalhado por processo.

Etapa 12 – Relatório final do processo root

O processo root deverá gerar um relatório consolidado da análise distribuída.

O relatório final deve conter

Informações gerais

- tamanho da imagem
- quantidade de processos MPI utilizada
- limiares utilizados
- tempo total da execução

Estatísticas globais

- total de pixels analisados
- intensidade média global
- maior intensidade global
- total de pixels suspeitos

- total de pixels altamente suspeitos
- percentual da imagem classificado como suspeito

Comparação entre os lados

- total de suspeitos no pulmão esquerdo
- total de suspeitos no pulmão direito
- qual lado apresentou maior concentração de áreas suspeitas

Estatísticas por processo

Para cada processo:

- faixa de linhas analisadas
- número de pixels suspeitos
- classificação da faixa
- intensidade máxima local

Classificação geral do exame simulado

O root deve definir uma classificação final, como por exemplo:

- sem indícios relevantes
- atenção clínica
- exame com alta concentração de áreas suspeitas

Essa classificação deve ser construída com base nas métricas globais, e não de forma aleatória.

ENTREGA

- Código-fonte
    - Executar com diferentes quantidades de processos, por exemplo:
    - 2 processos
    - 4 processos
    - 8 processos
- Responder às questões propostas com base nas execuções realizadas.

Questões de reflexão sobre a execução do sistema

Responda às questões abaixo com base nas execuções realizadas do programa, utilizando diferentes configurações de tamanho de imagem e número de processos MPI.

1 – Impacto do número de processos no tempo de execução

Execute o programa utilizando diferentes quantidades de processos MPI, por exemplo:

- 2 processos
- 4 processos
- 8 processos

Compare os tempos de execução obtidos.

O tempo total da aplicação diminuiu proporcionalmente ao aumento do número de processos? Explique o comportamento observado e discuta possíveis fatores que influenciam esse resultado, como custo de comunicação, sincronização e divisão de dados.

2 – Influência de processos mais lentos no cluster

Ative a simulação de atraso artificial em alguns processos (por exemplo utilizando sleep em processos com rank ímpar).

Observe o tempo total da execução.

Explique o que aconteceu com o desempenho da aplicação e por que um único processo mais lento pode afetar o tempo total da análise distribuída.

3 – Papel do MPI_Barrier na execução

Durante a execução do programa, identifique em quais momentos o uso do MPI_Barrier() influencia o comportamento do sistema.

Explique o que acontece quando um processo chega ao Barrier antes dos outros e como isso afeta a continuidade da execução do programa.

4 – Diferença entre Broadcast e Scatter neste projeto

No início do programa, os parâmetros da análise são enviados a todos os processos usando MPI_Bcast, enquanto a radiografia é dividida entre os processos usando MPI_Scatter.

Explique por que faz sentido utilizar Broadcast para os parâmetros e Scatter para a imagem. Discuta também o que aconteceria se toda a imagem fosse enviada para todos os processos usando apenas Broadcast.

5 – Diferença prática entre Reduce e Gather

Neste projeto foram utilizados dois mecanismos diferentes de coleta de informações:

- MPI_Reduce, para consolidar valores numéricos globais
- MPI_Gather, para reunir relatórios locais de cada processo

Explique por que cada um desses mecanismos foi utilizado em situações diferentes e quais seriam as dificuldades se apenas um deles fosse usado para tudo.

6 – Vantagem real do paralelismo para imagens pequenas

Execute o programa utilizando uma radiografia menor (por exemplo 500 × 500 pixels) e compare com execuções utilizando imagens maiores.

Discuta se o uso de MPI ainda traz vantagens para imagens pequenas e explique o motivo com base no custo de comunicação entre processos.

7 – Distribuição de áreas suspeitas na radiografia

Utilizando o relatório final gerado pelo programa, identifique:

- qual lado do pulmão apresentou maior quantidade de pixels suspeitos
- qual lado apresentou maior quantidade de pixels altamente suspeitos

Explique como o sistema chegou a essa conclusão e qual informação da matriz foi utilizada para separar pulmão esquerdo e pulmão direito.

8 – Análise da distribuição do trabalho entre os processos

Observe os relatórios individuais coletados com MPI_Gather.

Compare:

- número de pixels analisados por processo
- quantidade de suspeitos encontrados em cada faixa da imagem

Explique se o trabalho foi distribuído de forma equilibrada entre os processos ou se algumas regiões da imagem apresentaram maior concentração de alterações.

9 – Divisão da imagem quando o número de linhas não é divisível

Considere um cenário em que o número de linhas da imagem não seja divisível exatamente pelo número de processos MPI.

Explique qual estratégia você utilizou para resolver esse problema e discuta se essa solução pode gerar algum tipo de desequilíbrio na carga de trabalho entre os processos.